SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I KISHORE KUMAR V(192521371)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Kishore V
Signature of the student:

Annexure A

Scenario Based Assignment – Answers

1. Emergency Medicine Delivery Drones

Scenario: AI drones deliver medicines in a flood-affected region with changing roads, weather-dependent costs, a straight-line heuristic and partial real-time updates.

i. Greedy Best-First Search (GBFS)

GBFS expands the node with the lowest heuristic value h(n), i.e., the node that looks closest to the destination by straight-line distance. It ignores the cost already spent (g(n)).

Strengths:

- Very fast and uses little computation, so decisions are made quickly in an emergency.
- Simple to implement and works well when the straight-line estimate is close to reality.

Weaknesses under uncertainty:

- It ignores terrain and weather costs, so it may choose a short-looking but dangerous or costly path.
- It can be led into blocked areas or dead ends (flooded roads) because straight-line distance does not reflect obstacles.
- It is neither optimal nor complete in general and can get stuck in loops without a visited list.

ii. A* Search

A* uses f(n) = g(n) + h(n), where g(n) is the actual cost travelled so far (including terrain and weather costs) and h(n) is the straight-line estimate to the goal.

g(n) keeps the drone from choosing expensive or risky routes, while h(n) pulls the search towards the goal. This balance

explores fewer nodes than uniform cost search and still finds the least-cost route when h(n) is admissible (never overestimates). Straight-line distance is admissible, so A* is optimal and complete here, provided the cost updates are reflected in g(n).

iii. Impact of Heuristic Accuracy

- GBFS depends completely on the heuristic. A poor or misleading heuristic (for example, a straight line crossing a flooded river) sends it in the wrong direction with no correction from past cost.
- A* is more robust: g(n) corrects inaccuracies. A more accurate (closer to true cost, still admissible) heuristic makes A* expand fewer nodes. A weak heuristic (h = 0) makes it behave like uniform cost search – still optimal but slower.
- If the heuristic overestimates, A* may lose optimality.

iv. Effect of Dynamic Changes (Blocked Paths)

- When a path becomes blocked, the stored plan becomes invalid and the search must be re-run or repaired from the current position.
- GBFS recovers quickly because it is cheap, but may repeatedly choose bad routes.
- A* is more expensive per re-plan as the open list must be updated, but yields safer routes. Variants such as D* Lite or Lifelong Planning A* reuse earlier search effort and re-plan efficiently when edge costs change.
- Frequent changes increase re-planning time, so the search must be combined with periodic updates from sensors.

v. Recommendation

A* (preferably an incremental version such as D* Lite) is recommended.

- Optimality: it minimises total cost including terrain and weather.
- Safety: g(n) penalises risky or high-cost areas, avoiding dangerous routes.
- Real-time: with an admissible heuristic and re-planning only when updates arrive, delay stays acceptable; GBFS may be used only as a quick fallback when computation time is extremely limited.

2. Smart City Traffic Signal Optimisation

Scenario: AI adjusts signal timings across hundreds of intersections to minimise waiting time, fuel use and congestion in a huge, changing search space.

Problem Formulation

- State: the set of signal timings (cycle length, green split, offset) for all intersections.
- Objective function: minimise Cost = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion level).
- Operators: increase or decrease a green time, shift an offset, change a phase order.
- Constraints: minimum green time, pedestrian crossing time, safety clearance.

Traditional (uninformed or exhaustive) search is inefficient because the number of timing combinations grows exponentially with intersections, the traffic changes constantly so a stored solution quickly becomes outdated, and only the best final configuration matters, not the path to it.

i. Hill Climbing

Hill Climbing starts with the current signal plan, generates neighbouring plans (small timing changes), moves to the neighbour that lowers the cost most, and stops when no neighbour is better.

Limitations: it needs very little memory and is fast, but it can stop at a local optimum, has no way to go downhill to escape, depends on the starting plan and is not complete or optimal.

ii. Local Maxima, Plateaus and Ridges

- Local maximum (local optimum): a plan better than all neighbours but worse than the best global plan, e.g., one junction is perfectly tuned but it blocks the next junction, so the network still suffers.
- Plateau: a flat region where small timing changes do not alter the cost, e.g., off-peak hours where adding 2 seconds of green makes no difference, so the search wanders without direction.
- Ridge: a narrow path of improvement that needs several coordinated changes together, e.g., a 'green wave' along an avenue needs offsets at many junctions changed together; changing one at a time shows no improvement.

iii. Simulated Annealing and Genetic Algorithms

- Simulated Annealing accepts worse moves with probability $e^{(-\Delta E/T)}$. At high temperature T it explores widely, and as T is lowered it settles into a good solution. This lets it escape local optima and plateaus.
- Genetic Algorithm keeps a population of signal plans, selects the fittest (low waiting time and fuel), combines them by crossover and adds random mutation. It searches many regions in parallel and can handle ridges because crossover can change several junctions together.

iv. Recommendation

A Genetic Algorithm (possibly combined with simulated annealing for local refinement) is recommended.

- Scalability: it works on large, high-dimensional search spaces and can be run in parallel across zones of the city.
- Adaptability: the population can be re-evolved from the current best plans when traffic changes, so it adapts quickly without starting over.
- Simulated annealing is simpler and also good for a single zone, but a population-based method handles the whole-city scale better.

3. Autonomous Mars Rover

Scenario: A rover explores unknown terrain, avoiding craters and rocks, with limited sensing and no complete map.

i. Why an Online Search Agent

An offline search agent plans the whole route before acting, which needs a full, correct model of the environment. The rover does not have a map, so it cannot plan ahead completely. An online search agent interleaves planning and acting: it takes an action, observes what is sensed, updates its knowledge and then decides the next step. This is the only practical approach in an unknown environment.

ii. Challenges of Partial Observability and Dynamic Environments

- Only nearby terrain is visible, so hazards beyond the sensor range are unknown.
- Sensor noise and limited accuracy can give wrong readings about rocks or slopes.
- Actions may have uncertain outcomes (wheel slip on sand), and the same observation may correspond to different states.

- Dynamic factors such as dust, changing light or shifting sand change what the rover sees.
- Irreversible actions (falling into a crater) mean exploration must be safe; the rover must also avoid loops and dead ends.

iii. Building and Updating the Internal Model

- The rover keeps a map (for example, an occupancy grid) of visited cells, free cells, obstacles and hazards, along with the results of its actions.
- After each move it senses the surroundings, adds new cells, and corrects earlier beliefs (for example, with a Bayesian or probabilistic update).
- It remembers visited states and the cost of reaching them, so it can backtrack and avoid repeating, and it re-plans the path with the updated map.

iv. Comparison of Online Search with Uninformed and Informed Search

- Uninformed search (BFS, DFS, UCS): needs a known state space and uses no goal knowledge; it is wasteful and unsafe in an unknown environment.
- Informed search (A*, greedy): uses a heuristic and is efficient, but still assumes a known map and successor states in advance.
- Online search (for example, LRTA*, online DFS): expands only the current node, learns as it moves, and updates heuristic values with experience. It can work with unknown terrain, though it may not find the optimal path and can take extra steps.

v. Most Suitable Approach

An online search agent such as LRTA* (learning real-time A*) or online A* with map updating is most suitable.

- Uncertainty: it decides using only what is currently known and learns from new observations.
- Adaptability: it re-plans whenever new obstacles are sensed.
- Safety: with a cost function that penalises hazards and careful, step-by-step moves, risky areas are avoided; it is complemented by safe-state rules such as never entering unexplored cells next to a hazard.

4. University Exam Timetable Generation

Scenario: Generate an exam timetable that satisfies many constraints for a growing number of courses and students.

CSP Formulation

A Constraint Satisfaction Problem consists of variables, domains and constraints, and a solution is a complete assignment that violates no constraint.

i. Variables, Domains and Constraints

- Variables: each exam, E1, E2, …, En (and, optionally, its hall assignment).
- Domains: the possible (day, time slot, hall) combinations, for example 10 days × 3 slots × available halls. The domain may be reduced for faculty availability or special accommodations.
- Constraints:
- No overlap: two exams taken by the same student cannot share the same time slot.
- Hall capacity: the number of exams in a slot cannot exceed the number of halls, and students must fit the hall size.

- Precedence: if subject A must be before B, then time(A) < time(B).
- Faculty: an invigilator or examiner cannot be in two places at the same time or be assigned on unavailable days.
- Accommodations: students with special needs get extra time, a separate room or specific slots.

ii. Backtracking Search

Backtracking assigns a value to one exam at a time (depth-first), checks the constraints after every assignment, and if a conflict arises it undoes the last assignment (backtracks) and tries another value. It continues until all exams are scheduled or all options are exhausted. Heuristics such as MRV (choose the exam with the fewest remaining slots), degree heuristic and least-constraining-value improve it, but plain backtracking can still be slow for large problems.

iii. Constraint Propagation (Forward Checking)

After each assignment, forward checking removes inconsistent values from the domains of unassigned exams. For example, when Exam A is placed in Slot 1, that slot is removed from the domain of every exam sharing students with A. If any domain becomes empty, the search backtracks immediately instead of discovering the failure much later. Stronger techniques such as arc consistency (AC-3) prune even more. This cuts the search space and reduces the number of backtracks.

iv. Real-World Challenges and Best Strategy

Challenges: a very large number of exams and students, conflicting or last-minute changes, soft preferences (fairness, spread of exams), and possibly no perfect solution.

Best strategy: combine backtracking with MRV, degree and least-constraining-value heuristics and forward checking or arc consistency, and use local search (min-conflicts) or

decomposition by department to repair and scale to large timetables. Hard constraints must always be met, and soft ones can be optimised after a feasible schedule is found.

5. Real-Time Strategy Game AI

Scenario: An AI opponent must compete with human players, decide within strict time limits and cope with a very large game tree.

i. Minimax Modelling

The game is modelled as a two-player, zero-sum game tree. Nodes are game states, edges are moves, and leaves have utility values. MAX (the AI) chooses the move with the highest value, and MIN (the human) is assumed to choose the move that is worst for the AI. Values are backed up from the leaves: MAX nodes take the maximum of their children, and MIN nodes take the minimum. The root move with the best backed-up value is played, giving the best result against an optimal opponent.

ii. Computational Challenges

- The tree size is O(b^d), where b is the branching factor and d the depth, so it grows exponentially.
- Real-time games have many units and actions per turn, so b is very large.
- Evaluating every state in the time limit is impossible, so the search must be depth-limited.
- Memory use, repeated states and the need for quick responses add further difficulty.

iii. Alpha-Beta Pruning

Alpha is the best value MAX can already guarantee on the path, and beta is the best value MIN can guarantee. When a node's value shows that beta ≤ alpha, the remaining children

cannot affect the final decision, so they are pruned (not evaluated).

Reasoning: if MIN already has a reply that gives the AI less than an alternative MAX can obtain elsewhere, MAX will never choose that branch, so exploring it further is pointless.

Effect: the result is identical to minimax, but with good move ordering the best-case complexity falls from O(b^d) to O(b^(d/2)), allowing roughly twice the search depth in the same time. With bad ordering there is little gain.

iv. Evaluation Function

Because the full tree cannot be searched, a depth-limited search uses an evaluation function such as:

Eval(s) = w1·(material or unit strength difference) + w2·(resource income) + w3·(map/territory control) + w4·(position and safety of key units or base) + w5·(technology or upgrade advantage) + w6·(mobility or threats).

Justification: material and resources show the ability to fight and build; territory and position show strategic advantage; safety of the base reflects the risk of losing; weights are tuned from experience or learning. The function must be fast to compute and correlated with the actual chance of winning.

v. Balancing Optimality and Real-Time Performance

- Use alpha-beta pruning with good move ordering (best or killer moves first).
- Apply iterative deepening with a time limit, so the best move found so far is always available.
- Cut off at a depth limit and use the evaluation function; add a quiescence search for unstable positions.

- Use transposition tables to avoid repeated work, and heuristic or selective pruning of poor moves.

This gives near-optimal moves within strict time limits, accepting small loss of optimality for speed.